

DiffuChannel: comparison of real and diffusion-generated single-channel records

Barrett-Jolley lab

06 August 2026

Contents

Purpose	2
Packages	2
Configuration	2
Theme	3
Helper functions	4
Reading records	4
Windowing and MMD	5
Welch power spectral density	6
Dwell times and a simple idealiser	6
Subsampled KS test	7
1. Load the data	7
Table 1: seed data summary	9
2. Figure 2: traces and amplitude distribution	9
2B–D: example traces	9
2E: all-points amplitude histograms	11
3. Maximum mean discrepancy	13
4. Figure 3: noise structure	14
3A: power spectral density of the closed state	14
3B: autocorrelation of the baseline residual	15
5. Figure 4: the slow envelope	17

6. Figure 5: block joins	19
7. Figure 6: dwell times and the ground-truth check	21
8. Figure 7: manifold projections	23
9. Table 3: collected similarity metrics	25
10. Write the placeholder values	25
Session information	27

Purpose

This notebook takes real (seed) patch-clamp records and DiffuChannel-generated records, computes every similarity metric reported in the manuscript, draws every figure, and writes a `results_values.csv` whose rows are keyed to the `[[placeholder]]` names used in the manuscript draft.

Nothing here assumes a particular result. If the model has not learned the noise structure, the plots will say so.

Packages

```
required <- c("dplyr", "tidyr", "ggplot2", "readr", "purrr", "arrow",
             "scales", "patchwork", "uwot", "Rtsne")

missing <- required[!vapply(required, requireNamespace, logical(1), quietly = TRUE)]
if (length(missing)) {
  message("Install these before running: ", paste(missing, collapse = ", "))
  message('install.packages(c(" ', paste(missing, collapse = '"', "'"), '"))')
}

library(dplyr)
library(tidyr)
library(ggplot2)
library(readr)
library(purrr)
library(arrow)
library(patchwork)

set.seed(42)
```

Configuration

Everything you need to change lives in this block.

```

CFG <- list(

  # ---- where the data are -----
  real_dir      = "./pages",          # folder of seed CSV / Parquet
  gen_file      = "./generated_dataset.parquet", # cascade output
  gen_single_file = "./generated_single_stage.parquet", # ablation; NA if not run
  out_dir       = "./figures",

  # ---- recording parameters -----
  sample_hz     = 1e4,      # acquisition rate, Hz. Must match SAMPLE_DT in diffusion_app.py
  filter_hz     = 1e3,      # anti-alias / Bessel corner frequency, Hz

  # ---- must match how the model was trained -----
  seq_len       = 128,      # training window, samples
  slow_factor   = 64,      # decimation factor of the coarse stage
  context_len   = 512,      # carried context at block joins
  block_len     = 32768,    # generation block length
  burst_tc      = 1000,     # BURST_TC in diffusion_app.py: flicker threshold, samples
  emb_dim       = 4,        # per-record embedding width used by the coarse stage
  emb_drop      = 0.1,      # embedding dropout during coarse training

  # ---- analysis knobs -----
  n_windows_mmd = 400,      # windows per group for MMD.  $O(n^2)$ : keep modest
  n_windows_proj = 600,    # windows per group for UMAP / t-SNE
  proj_win      = 512,      # window length for manifold projections
  psd_seg       = 1024,     # Welch segment length
  acf_lag_max   = 200,      # autocorrelation lag, samples
  ks_subsample  = 2e5,      # cap for KS tests (ks.test is slow on millions)

  # ---- plotting -----
  trace_secs    = 0.5,      # duration of example traces to plot
  base_size     = 10

)

dir.create(CFG$out_dir, showWarnings = FALSE, recursive = TRUE)

# Results are collected here and written out at the end.
RESULTS <- new.env(parent = emptyenv())
RESULTS$rows <- list()

record_value <- function(key, value, note = "") {
  RESULTS$rows[[key]] <- tibble::tibble(
    placeholder = key,
    value       = if (is.numeric(value)) format(value, digits = 4) else as.character(value),
    note        = note
  )
  invisible(value)
}

```

Theme

A plain publication theme.

```

theme_channel <- function(base_size = CFG$base_size) {
  theme_classic(base_size = base_size) +
    theme(
      strip.background = element_blank(),
      strip.text        = element_text(face = "bold", hjust = 0),
      legend.position   = "top",
      legend.title      = element_blank(),
      legend.key.size    = unit(0.8, "lines"),
      plot.title        = element_text(face = "bold", size = base_size + 1),
      axis.text         = element_text(colour = "black")
    )
}

pal_source <- c(
  "Real"          = "#1b1b1b",
  "Generated"     = "#c0392b",
  "Single-stage"  = "#2980b9",
  "White noise"   = "#7f8c8d"
)

save_fig <- function(plot, name, width = 7, height = 4.2) {
  ggsave(file.path(CFG$out_dir, paste0(name, ".pdf")), plot,
    width = width, height = height)
  ggsave(file.path(CFG$out_dir, paste0(name, ".png")), plot,
    width = width, height = height, dpi = 300)
  invisible(plot)
}

```

Helper functions

Reading records

Handles both the tidy column names and whatever the acquisition software produced.

```

normalise_names <- function(nms) {
  gsub("[_]", " ", trimws(tolower(nms)))
}

read_record <- function(path) {
  df <- if (grepl("\\.parquet$", path, ignore.case = TRUE)) {
    as.data.frame(arrow::read_parquet(path))
  } else {
    as.data.frame(readr::read_csv(path, show_col_types = FALSE))
  }

  nn <- normalise_names(names(df))
  idx <- function(target) {
    hit <- which(nn %in% target)
    if (length(hit)) hit[1] else NA_integer_
  }

  i_chan <- idx(c("channels", "channel", "ideal", "idealisation", "idealization"))
}

```

```

i_cur <- idx(c("noisy current", "current", "raw", "raw current", "i"))
i_time <- idx(c("time", "t"))

if (is.na(i_chan) || is.na(i_cur)) {
  stop("Could not find Channels / Noisy Current columns in ", basename(path),
       ". Found: ", paste(names(df), collapse = ", "))
}

out <- tibble::tibble(
  channels = as.numeric(df[[i_chan]]),
  current = as.numeric(df[[i_cur]])
)
out$time <- if (!is.na(i_time)) as.numeric(df[[i_time]]) else
  (seq_len(nrow(out)) - 1) / CFG$sample_hz

out |>
  dplyr::filter(is.finite(channels), is.finite(current)) |>
  dplyr::select(time, channels, current)
}

read_folder <- function(dir) {
  files <- c(
    list.files(dir, pattern = "\\..csv$", full.names = TRUE),
    list.files(dir, pattern = "\\..parquet$", full.names = TRUE)
  ) |> sort()

  if (!length(files)) stop("No CSV or Parquet files found in ", dir)

  purrr::map(files, ~ dplyr::mutate(read_record(.x), record = basename(.x)))
}

```

Windowing and MMD

The MMD estimator here mirrors the Python implementation in `diffusion_103.py` (biased V-statistic, Gaussian kernel, bandwidth $\gamma = 1 / L$) so that the numbers in the notebook and the numbers printed during training are on the same scale.

```

make_windows <- function(x, win, n_max = Inf, stride = NULL) {
  n <- length(x)
  if (n < win) return(matrix(numeric(0), nrow = 0, ncol = win))
  if (is.null(stride)) {
    stride <- max(1L, floor((n - win) / max(1L, min(n_max, n))))
  }
  starts <- seq(1L, n - win + 1L, by = stride)
  if (length(starts) > n_max) starts <- sample(starts, n_max)
  t(vapply(starts, function(s) x[s:(s + win - 1L)], numeric(win)))
}

sq_dists <- function(A, B) {
  aa <- rowSums(A^2)
  bb <- rowSums(B^2)
  d <- outer(aa, bb, "+") - 2 * tcrossprod(A, B)
}

```

```

  pmax(d, 0)
}

mmd_rbf <- function(X, Y, gamma = NULL) {
  if (nrow(X) < 2 || nrow(Y) < 2) return(NA_real_)
  if (is.null(gamma)) gamma <- 1 / ncol(X)
  k <- function(A, B) exp(-gamma * sq_dists(A, B))
  mean(k(X, X)) + mean(k(Y, Y)) - 2 * mean(k(X, Y))
}

```

Welch power spectral density

Written out rather than pulled from a package, so that the estimate is transparent and the notebook has one fewer dependency.

```

welch_psd <- function(x, fs, nperseg = 1024, noverlap = NULL, detrend = TRUE) {
  x <- as.numeric(x[is.finite(x)])
  if (length(x) < nperseg) return(NULL)
  if (is.null(noverlap)) noverlap <- nperseg %% 2

  step <- nperseg - noverlap
  starts <- seq(1L, length(x) - nperseg + 1L, by = step)
  w <- 0.5 - 0.5 * cos(2 * pi * (seq_len(nperseg) - 1) / (nperseg - 1)) # Hann
  U <- sum(w^2)

  acc <- numeric(nperseg %% 2 + 1)
  for (s in starts) {
    seg <- x[s:(s + nperseg - 1L)]
    if (detrend) seg <- seg - mean(seg)
    X <- stats::fft(seg * w)[seq_len(nperseg %% 2 + 1)]
    acc <- acc + (Mod(X)^2) / (fs * U)
  }
  psd <- acc / length(starts)
  psd[-c(1, length(psd))] <- 2 * psd[-c(1, length(psd))] # one-sided

  tibble::tibble(
    freq = seq(0, fs / 2, length.out = length(psd)),
    psd = psd
  ) |> dplyr::filter(freq > 0)
}

```

Dwell times and a simple idealiser

The half-amplitude threshold criterion is the classical approach and is what we use to re-idealise the *generated* current in Section 6, so that the missed-event losses reported there are the ones a conventional analysis would actually incur.

```

dwell_times <- function(ideal, fs) {
  r <- rle(as.integer(round(ideal)))
  tibble::tibble(
    level = r$values,

```

```

    n_sample = r$lengths,
    dwell_ms = 1000 * r$lengths / fs
  ) |>
  dplyr::filter(n_sample > 0) |>
  dplyr::mutate(state = ifelse(level == 0, "Closed", "Open"))
}

# Half-amplitude threshold idealiser. Levels are estimated from the amplitude
# histogram of the closed baseline and the mode of the open population.
idealise_half_amplitude <- function(current, unit_pA = NULL, n_levels = 1) {
  if (is.null(unit_pA)) {
    d <- density(current, n = 2048)
    peak <- d$x[which.max(d$y)]
    far <- current[abs(current - peak) > 3 * mad(current)]
    unit_pA <- if (length(far) > 50) median(far) - peak else NA_real_
  }
  if (!is.finite(unit_pA) || unit_pA == 0) {
    warning("Could not estimate unitary amplitude; supply unit_pA explicitly.")
    return(rep(NA_integer_, length(current)))
  }
  baseline <- median(current)
  lvl <- (current - baseline) / unit_pA
  pmin(pmax(as.integer(round(lvl)), 0L), n_levels)
}

block_mean <- function(x, factor) {
  factor <- max(1L, as.integer(factor))
  nb <- length(x) %/% factor
  if (nb < 1) return(mean(x))
  colMeans(matrix(x[seq_len(nb * factor)], nrow = factor))
}

```

Subsampled KS test

```

ks_safe <- function(a, b, n_max = CFG$ks_subsample) {
  a <- a[is.finite(a)]; b <- b[is.finite(b)]
  if (length(a) > n_max) a <- sample(a, n_max)
  if (length(b) > n_max) b <- sample(b, n_max)
  suppressWarnings(stats::ks.test(a, b))
}

```

1. Load the data

```

real_list <- read_folder(CFG$real_dir)
real <- dplyr::bind_rows(real_list)

```

```

gen <- read_record(CFG$gen_file)
gen$record <- "generated"

has_single <- !is.na(CFG$gen_single_file) && file.exists(CFG$gen_single_file)
gen_single <- if (has_single) {
  x <- read_record(CFG$gen_single_file); x$record <- "single-stage"; x
} else {
  message("No single-stage file found. Ablation sections will be skipped.")
  NULL
}

# ---- derive the true sample rate of each file from its Time column -----
# The app's export "Decimation factor" slider multiplies the Time step, so a
# generated file can sit at a different rate from the seed data. Everything
# below (PSD, ACF, dwell times) is wrong if that goes unnoticed, so take the
# rate from the data rather than from CFG.
fs_of <- function(d, fallback = CFG$sample_hz) {
  if (nrow(d) < 3) return(fallback)
  dt <- stats::median(diff(d$time), na.rm = TRUE)
  if (!is.finite(dt) || dt <= 0) return(fallback)
  1 / dt
}

FS_REAL <- fs_of(real_list[[1]])
FS_GEN <- fs_of(gen)
FS_SING <- if (has_single) fs_of(gen_single) else NA_real_

cat(sprintf("Real:      %d record(s), %s samples (%.1f s)\n",
            length(real_list), format(nrow(real), big.mark = ","),
            nrow(real) / FS_REAL))

## Real:      1 record(s), 824,001 samples (41.2 s)

cat(sprintf("Generated: %s samples (%.1f s)\n",
            format(nrow(gen), big.mark = ","), nrow(gen) / CFG$sample_hz))

## Generated: 100,000 samples (10.0 s)

cat(sprintf("Sample rate  real %.1f Hz | generated %.1f Hz\n", FS_REAL, FS_GEN))

## Sample rate  real 20000.0 Hz | generated 10000.0 Hz

if (abs(FS_REAL - FS_GEN) / FS_REAL > 0.01) {
  warning(sprintf(paste("Real and generated sample rates differ (%.1f vs %.1f Hz).",
                        "Almost certainly the export Decimation factor was > 1.",
                        "Per-file rates are used throughout, but a generated record",
                        "coarser than the seed data cannot reproduce its high-frequency",
                        "noise and the PSD comparison will be truncated."),
                  FS_REAL, FS_GEN))
}

```



```

record_value("n_records", length(real_list), "seed records")
record_value("n_samples", format(nrow(real), big.mark = ","), "total seed samples")
record_value("n_seconds", round(nrow(real) / FS_REAL, 1), "seconds of seed data")
record_value("sample_khz", FS_REAL / 1000)
record_value("filter_khz", CFG$filter_hz / 1000)
record_value("seq_len", CFG$seq_len)
record_value("slow_factor", CFG$slow_factor)
record_value("context_len", CFG$context_len)
record_value("block_len", CFG$block_len)
record_value("burst_tc", CFG$burst_tc, "flicker threshold for the burst clock")
record_value("emb_dim", CFG$emb_dim)
record_value("emb_drop", CFG$emb_drop)
record_value("seq_len_ms", round(1000 * CFG$seq_len / FS_REAL, 2),
             "training window duration")
record_value("long_len_s", round(nrow(gen) / FS_GEN, 1))
record_value("n_blocks", max(1, ceiling(nrow(gen) /
                                         (CFG$block_len - CFG$context_len))))

```

Table 1: seed data summary

```

table1 <- purrr::map_dfr(real_list, function(d) {
  tibble::tibble(
    Record      = d$record[1],
    Samples     = nrow(d),
    `Duration (s)` = round(nrow(d) / FS_REAL, 2),
    `Max levels` = max(d$channels, na.rm = TRUE),
    `Open prob.` = round(mean(d$channels > 0), 4),
    `Mean (pA)`  = round(mean(d$current), 3),
    `SD (pA)`    = round(sd(d$current), 3)
  )
})

knitr::kable(table1, caption = "Table 1. Seed records used for training and evaluation.")

```

Table 1: Table 1. Seed records used for training and evaluation.

Record	Samples	Duration (s)	Max levels	Open prob.	Mean (pA)	SD (pA)
claire_03560xoriginqubnoise1_SamForm824001	824001	41.2	1	0.3654	13.695	7.141

```

readr::write_csv(table1, file.path(CFG$out_dir, "table1_seed_data.csv"))

```

2. Figure 2: traces and amplitude distribution

2B–D: example traces

```

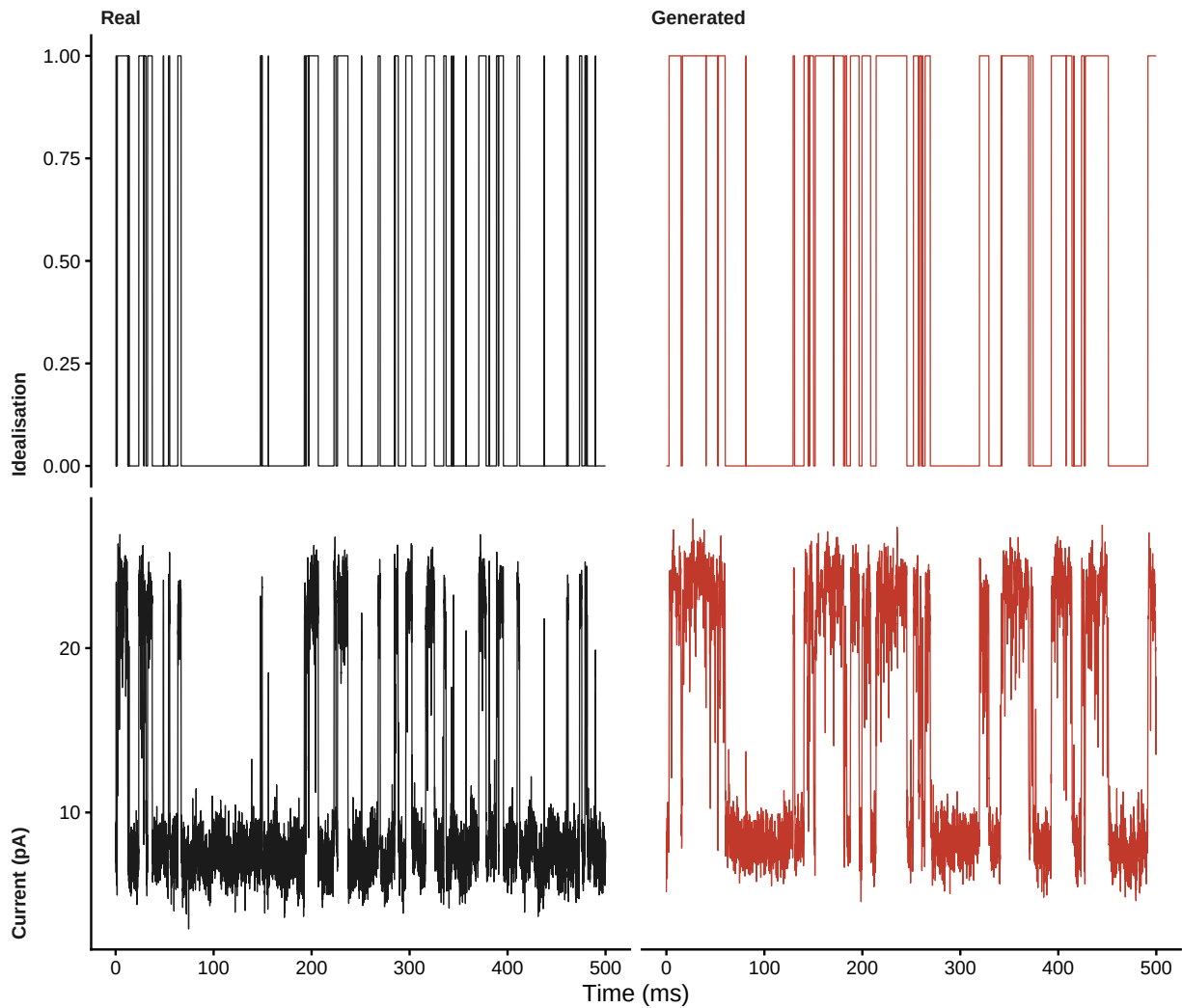
n_show_real <- round(CFG$trace_secs * FS_REAL)
n_show_gen  <- round(CFG$trace_secs * FS_GEN)

trace_df <- dplyr::bind_rows(
  real |> dplyr::slice_head(n = n_show_real) |>
    dplyr::select(time, channels, current) |> dplyr::mutate(source = "Real"),
  gen  |> dplyr::slice_head(n = n_show_gen) |>
    dplyr::select(time, channels, current) |> dplyr::mutate(source = "Generated")
) |>
  dplyr::mutate(
    time_ms = 1000 * (time - min(time)),
    source = factor(source, levels = c("Real", "Generated"))
  ) |>
  tidyr::pivot_longer(c(channels, current), names_to = "signal", values_to = "value") |>
  dplyr::mutate(signal = factor(
    signal, levels = c("channels", "current"),
    labels = c("Idealisation", "Current (pA)")
  ))

p_traces <- ggplot(trace_df, aes(time_ms, value, colour = source)) +
  geom_line(linewidth = 0.25) +
  facet_grid(signal ~ source, scales = "free_y", switch = "y") +
  scale_colour_manual(values = pal_source, guide = "none") +
  labs(x = "Time (ms)", y = NULL) +
  theme_channel() +
  theme(strip.placement = "outside")

p_traces

```



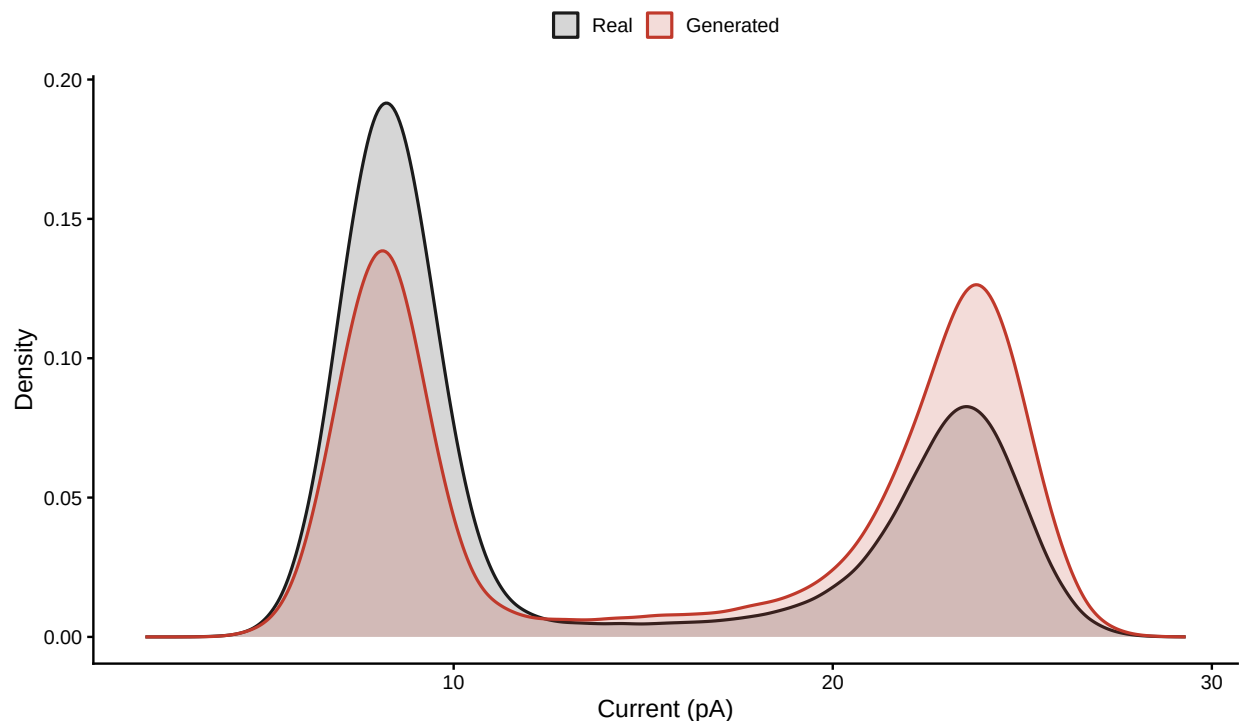
```
save_fig(p_traces, "fig2_traces", height = 5)
```

2E: all-points amplitude histograms

```
amp_df <- dplyr::bind_rows(
  tibble::tibble(current = real$current, source = "Real"),
  tibble::tibble(current = gen$current, source = "Generated")
) |>
  dplyr::mutate(source = factor(source, levels = c("Real", "Generated")))

p_amp <- ggplot(amp_df, aes(current, colour = source, fill = source)) +
  geom_density(alpha = 0.18, linewidth = 0.6, adjust = 0.6) +
  scale_colour_manual(values = pal_source) +
  scale_fill_manual(values = pal_source) +
  labs(x = "Current (pA)", y = "Density") +
  theme_channel()
```

p_amp



```
save_fig(p_amp, "fig2E_amplitude_histogram")

# Modes and dispersion
mode_of <- function(x) { d <- density(x, n = 4096); d$x[which.max(d$y)] }

closed_real <- real$current[real$channels == 0]
closed_gen <- gen$current[gen$channels == 0]

record_value("modes_real", round(mode_of(real$current), 3), "modal current, real, pA")
record_value("modes_gen", round(mode_of(gen$current), 3), "modal current, generated, pA")
record_value("sd_real", round(sd(closed_real), 4), "closed-state SD, real, pA")
record_value("sd_gen", round(sd(closed_gen), 4), "closed-state SD, generated, pA")

ks_amp <- ks_safe(real$current, gen$current)
record_value("ks_D", round(unname(ks_amp$statistic), 4),
             "KS statistic, all-points amplitude, real vs generated")
ks_amp
```

```
##
## Asymptotic two-sample Kolmogorov-Smirnov test
##
## data: a and b
## D = 0.1857, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

3. Maximum mean discrepancy

Three numbers, and the first is meaningless without the other two. The within-real value is the floor from finite sampling; the white-noise value is what a naive noise model scores.

```
W_real <- make_windows(real$current, CFG$seq_len, CFG$n_windows_mmd)
W_gen  <- make_windows(gen$current,  CFG$seq_len, CFG$n_windows_mmd)

half   <- floor(nrow(real) / 2)
W_r1   <- make_windows(real$current[1:half],          CFG$seq_len, CFG$n_windows_mmd)
W_r2   <- make_windows(real$current[(half + 1):nrow(real)], CFG$seq_len, CFG$n_windows_mmd)

white  <- rnorm(nrow(real), mean(real$current), sd(real$current))
W_white <- make_windows(white, CFG$seq_len, CFG$n_windows_mmd)

mmd_main      <- mmd_rbf(W_real, W_gen)
mmd_within_real <- mmd_rbf(W_r1,  W_r2)
mmd_white     <- mmd_rbf(W_real, W_white)

mmd_tbl <- tibble::tibble(
  Comparison = c("Real vs generated", "Real vs real (split half)",
                 "Real vs Gaussian white noise"),
  MMD        = c(mmd_main, mmd_within_real, mmd_white)
)

if (has_single) {
  W_gs <- make_windows(gen_single$current, CFG$seq_len, CFG$n_windows_mmd)
  mmd_tbl <- dplyr::bind_rows(
    mmd_tbl,
    tibble::tibble(Comparison = "Real vs single-stage", MMD = mmd_rbf(W_real, W_gs))
  )
}

knitr::kable(mmd_tbl, digits = 6, caption = "Maximum mean discrepancy, Gaussian kernel.")
```

Table 2: Maximum mean discrepancy, Gaussian kernel.

Comparison	MMD
Real vs generated	0.006371
Real vs real (split half)	0.007015
Real vs Gaussian white noise	0.013169

```
record_value("mmd_main",      signif(mmd_main, 4))
record_value("mmd_within_real", signif(mmd_within_real, 4))
record_value("mmd_white",     signif(mmd_white, 4))
```

4. Figure 3: noise structure

3A: power spectral density of the closed state

Computed on closed-state segments only, so gating does not contaminate the estimate. This is the plot that tells you whether the model learned the recording noise or just its variance.

```
longest_closed_run <- function(d, min_len) {
  r <- rle(d$channels == 0)
  ends <- cumsum(r$lengths)
  starts <- ends - r$lengths + 1
  keep <- which(r$values & r$lengths >= min_len)
  if (!length(keep)) return(numeric(0))
  unlist(lapply(keep, function(i) d$current[starts[i]:ends[i]]))
}

closed_real_seg <- longest_closed_run(real, CFG$psd_seg)
closed_gen_seg <- longest_closed_run(gen, CFG$psd_seg)

psd_labelled <- function(x, label, fs) {
  p <- welch_psd(x, fs, CFG$psd_seg)
  if (is.null(p)) {
    warning("Not enough closed-state samples for a PSD estimate: ", label,
           ". Lower CFG$psd_seg or use a record with longer closed periods.")
    return(NULL)
  }
  dplyr::mutate(p, source = label)
}

psd_df <- dplyr::bind_rows(
  psd_labelled(closed_real_seg, "Real", FS_REAL),
  psd_labelled(closed_gen_seg, "Generated", FS_GEN)
)

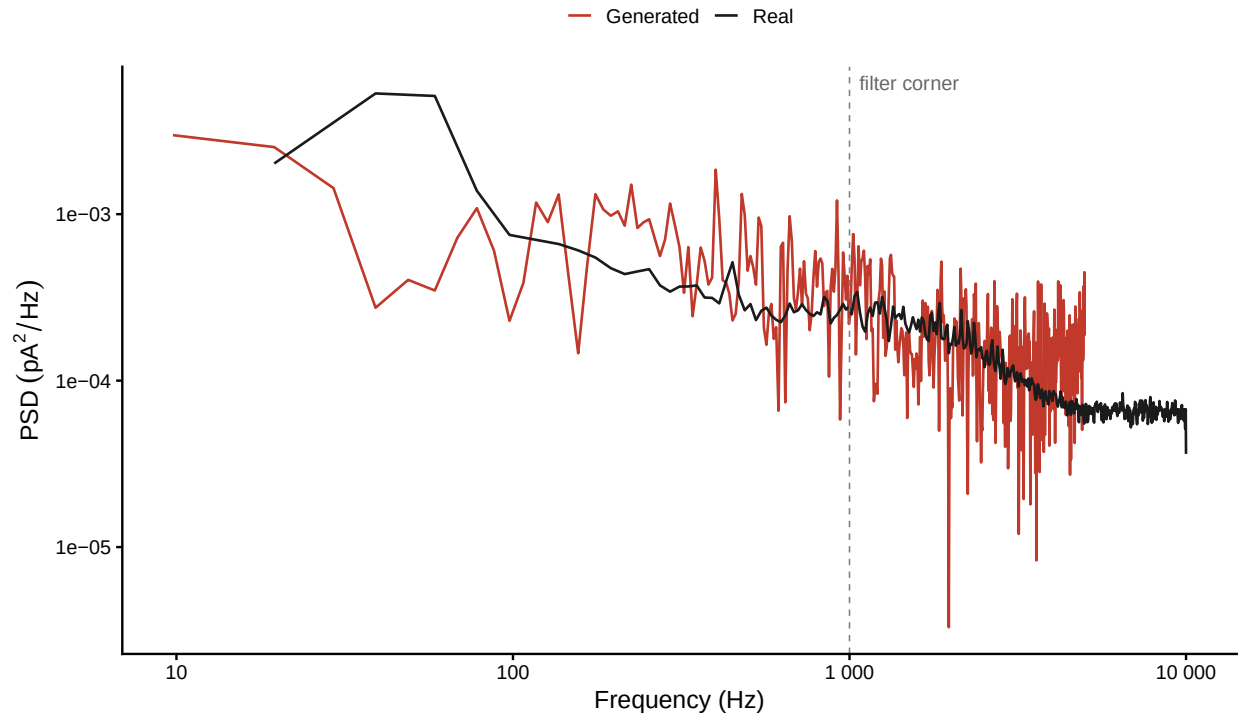
stopifnot(nrow(psd_df) > 0)

if (has_single) {
  cs <- longest_closed_run(gen_single, CFG$psd_seg)
  if (length(cs) >= CFG$psd_seg) {
    psd_df <- dplyr::bind_rows(psd_df, psd_labelled(cs, "Single-stage", FS_SING))
  }
}

p_psd <- ggplot(psd_df, aes(freq, psd, colour = source)) +
  geom_line(linewidth = 0.5) +
  geom_vline(xintercept = CFG$filter_hz, linetype = "dashed",
            colour = "grey50", linewidth = 0.3) +
  annotate("text", x = CFG$filter_hz, y = Inf, label = "filter corner",
         hjust = -0.1, vjust = 1.6, size = 2.8, colour = "grey40") +
  scale_x_log10(labels = scales::label_number()) +
  scale_y_log10(labels = scales::label_scientific()) +
  scale_colour_manual(values = pal_source) +
  labs(x = "Frequency (Hz)", y = expression(PSD~(pA^2/Hz))) +
```

```
theme_channel()
```

```
p_psd
```



```
save_fig(p_psd, "fig3A_psd")
```

```
# Quantify agreement as the RMS difference in log10 PSD below the filter corner.
psd_cmp <- psd_df |>
  dplyr::filter(freq < CFG$filter_hz, source %in% c("Real", "Generated")) |>
  tidyr::pivot_wider(names_from = source, values_from = psd) |>
  tidyr::drop_na()

psd_rmse <- sqrt(mean((log10(psd_cmp$Real) - log10(psd_cmp$Generated))^2))
record_value("psd_log_rmse", round(psd_rmse, 3),
  "RMS difference in log10 PSD below filter corner")
record_value("psd_description",
  sprintf(paste("Real and generated spectra agree to an RMS difference of %.2f",
    "log10 units below the %g Hz filter corner."),
    psd_rmse, CFG$filter_hz),
  "REPLACE with real reading of Figure 3A")
```

3B: autocorrelation of the baseline residual

```
acf_of <- function(x, lag_max, fs) {
  a <- stats::acf(x - mean(x), lag.max = lag_max, plot = FALSE, demean = TRUE)
  tibble::tibble(lag_ms = 1000 * a$lag[, 1, 1] / fs, acf = a$acf[, 1, 1])
}
```

```

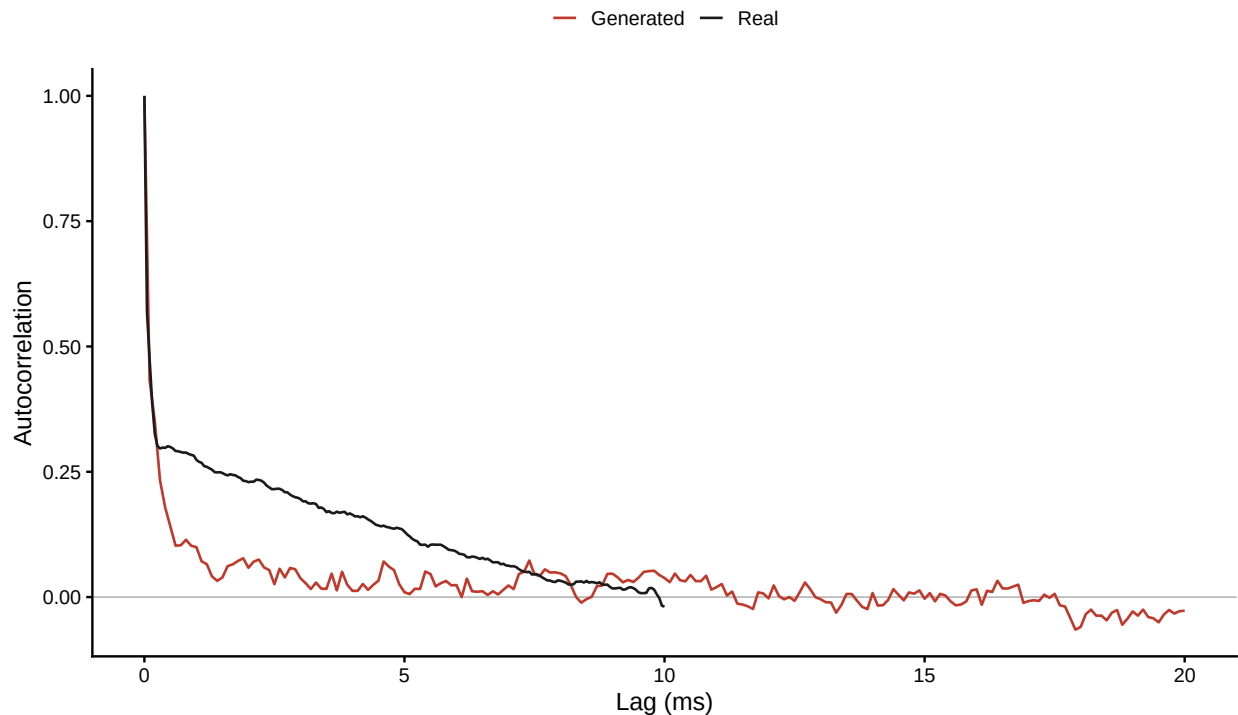
}

acf_df <- dplyr::bind_rows(
  dplyr::mutate(acf_of(closed_real_seg, CFG$acf_lag_max, FS_REAL), source = "Real"),
  dplyr::mutate(acf_of(closed_gen_seg, CFG$acf_lag_max, FS_GEN), source = "Generated")
)

p_acf <- ggplot(acf_df, aes(lag_ms, acf, colour = source)) +
  geom_hline(yintercept = 0, colour = "grey70", linewidth = 0.3) +
  geom_line(linewidth = 0.5) +
  scale_colour_manual(values = pal_source) +
  labs(x = "Lag (ms)", y = "Autocorrelation") +
  theme_channel()

p_acf

```



```

save_fig(p_acf, "fig3B_autocorrelation")

# Compare on lag index rather than lag time, so a rate mismatch does not
# silently drop every row when the two lag grids fail to line up.
acf_wide <- acf_df |>
  dplyr::group_by(source) |>
  dplyr::mutate(lag_n = dplyr::row_number()) |>
  dplyr::ungroup() |>
  dplyr::select(lag_n, source, acf) |>
  tidyr::pivot_wider(names_from = source, values_from = acf) |>
  tidyr::drop_na()
acf_rmse <- sqrt(mean((acf_wide$Real - acf_wide$Generated)^2))

```



```

record_value("acf_rmse", round(acf_rmse, 4), "RMS difference in ACF")
record_value("acf_description",
  sprintf("ACF curves differ by an RMS of %.3f over the first %.1f ms of lag.",
    acf_rmse, 1000 * CFG$acf_lag_max / FS_REAL),
  "REPLACE with real reading of Figure 3B")

p_noise <- p_psd + p_acf + patchwork::plot_annotation(tag_levels = "A")
save_fig(p_noise, "fig3_noise", width = 10, height = 4)

```

5. Figure 4: the slow envelope

```

env_real <- block_mean(real$current, CFG$slow_factor)
env_gen <- block_mean(gen$current, CFG$slow_factor)

env_len <- min(length(env_real), length(env_gen))
env_df <- dplyr::bind_rows(
  tibble::tibble(idx = seq_len(env_len), value = env_real[seq_len(env_len)],
    source = "Real"),
  tibble::tibble(idx = seq_len(env_len), value = env_gen[seq_len(env_len)],
    source = "Generated")
)

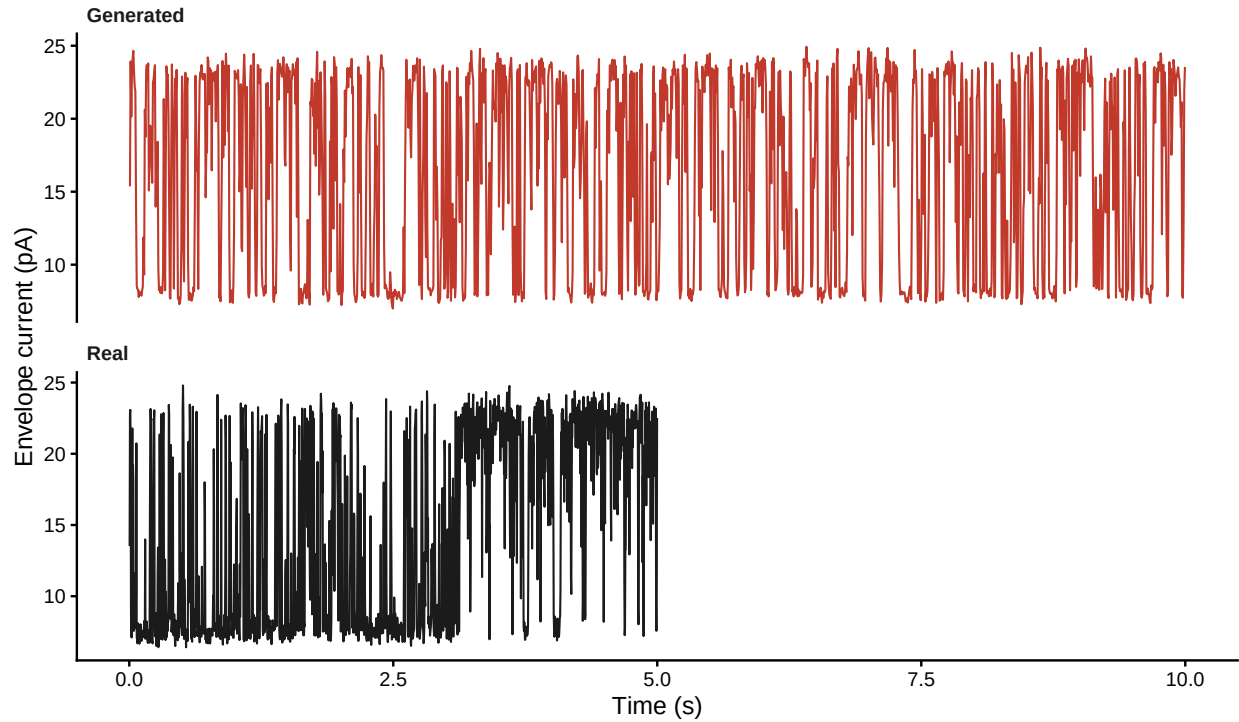
if (has_single) {
  env_single <- block_mean(gen_single$current, CFG$slow_factor)
  e1 <- min(env_len, length(env_single))
  env_df <- dplyr::bind_rows(
    env_df,
    tibble::tibble(idx = seq_len(e1), value = env_single[seq_len(e1)],
      source = "Single-stage")
  )
}

env_fs <- c("Real" = FS_REAL, "Generated" = FS_GEN, "Single-stage" = FS_SING)
env_df <- dplyr::mutate(env_df,
  time_s = idx * CFG$slow_factor / unname(env_fs[source]))

p_env <- ggplot(env_df, aes(time_s, value, colour = source)) +
  geom_line(linewidth = 0.4) +
  facet_wrap(~ source, ncol = 1, scales = "free_y") +
  scale_colour_manual(values = pal_source, guide = "none") +
  labs(x = "Time (s)", y = "Envelope current (pA)") +
  theme_channel()

p_env

```



```

save_fig(p_env, "fig4A_envelope", height = 5)

p_env_dist <- ggplot(env_df, aes(value, colour = source, fill = source)) +
  geom_density(alpha = 0.18, linewidth = 0.5) +
  scale_colour_manual(values = pal_source) +
  scale_fill_manual(values = pal_source) +
  labs(x = "Envelope current (pA)", y = "Density") +
  theme_channel()

save_fig(p_env_dist, "fig4C_envelope_distribution")

record_value("env_sd_real", round(sd(env_real), 4))
record_value("env_sd_cascade", round(sd(env_gen), 4))

We_r <- make_windows(env_real, min(64, length(env_real)), CFG$n_windows_mmd)
We_g <- make_windows(env_gen, min(64, length(env_gen)), CFG$n_windows_mmd)
record_value("mmd_env_cascade", signif(mmd_rbf(We_r, We_g), 4))

if (has_single) {
  record_value("env_sd_single", round(sd(env_single), 4))
  We_s <- make_windows(env_single, min(64, length(env_single)), CFG$n_windows_mmd)
  record_value("mmd_env_single", signif(mmd_rbf(We_r, We_s), 4))
} else {
  record_value("env_sd_single", "NOT RUN", "no single-stage ablation available")
  record_value("mmd_env_single", "NOT RUN", "no single-stage ablation available")
}

```

6. Figure 5: block joins

If the context-carry inpainting works, the distribution of sample-to-sample differences in the neighbourhood of a block boundary should be indistinguishable from that within blocks. If it does not work, you will see it here before a reviewer does.

```
step <- CFG$block_len - CFG$context_len
boundaries <- seq(step, nrow(gen) - 1, by = step)
boundaries <- boundaries[boundaries > CFG$context_len &
                        boundaries < (nrow(gen) - CFG$context_len)]

diffs <- diff(gen$current)

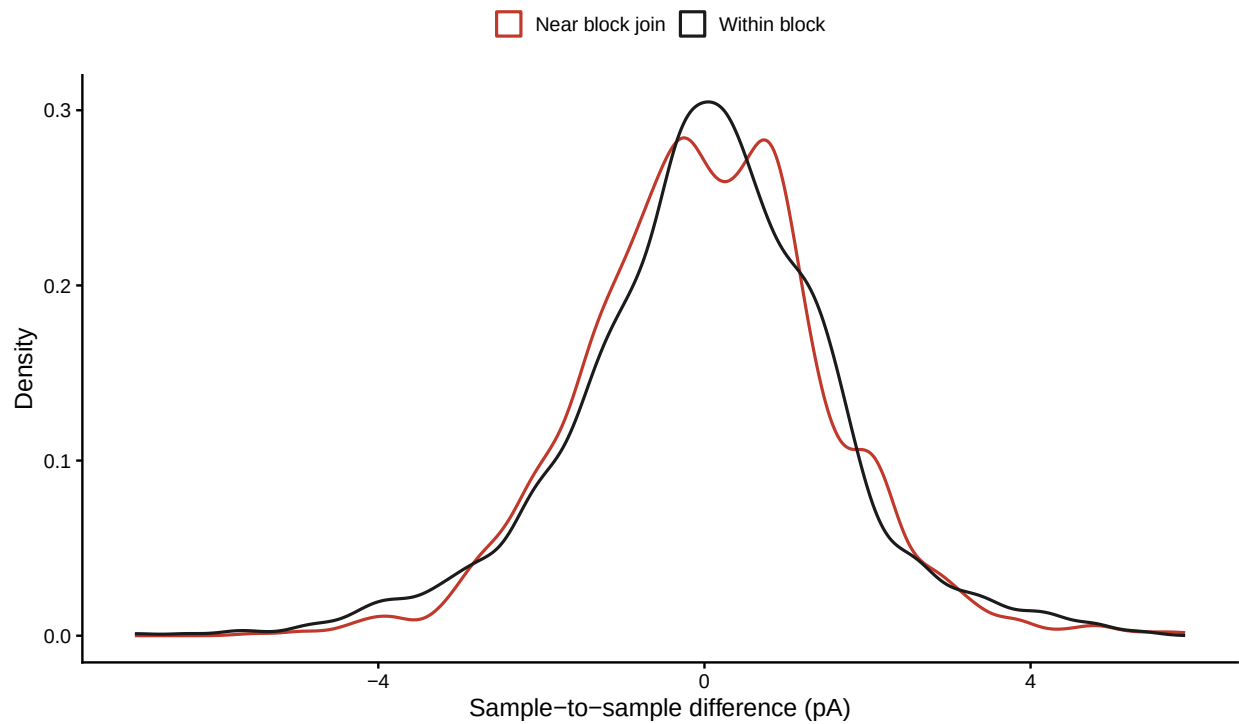
if (length(boundaries)) {
  near_idx <- unique(unlist(lapply(boundaries, function(b)
    seq(b - CFG$context_len %/% 2, b + CFG$context_len %/% 2))))
  near_idx <- near_idx[near_idx >= 1 & near_idx <= length(diffs)]
  far_idx <- setdiff(seq_along(diffs), near_idx)

  ks_seam <- ks_safe(diffs[near_idx], diffs[far_idx])
  record_value("ks_seam", round(unnamed(ks_seam$statistic), 4),
    "KS statistic, sample differences near vs away from block joins")
  print(ks_seam)

  seam_df <- dplyr::bind_rows(
    tibble::tibble(d = diffs[near_idx], where = "Near block join"),
    tibble::tibble(d = diffs[sample(far_idx, min(length(far_idx),
      length(near_idx)))],
      where = "Within block")
  )

  p_seam <- ggplot(seam_df, aes(d, colour = where)) +
    geom_density(linewidth = 0.6, adjust = 0.8) +
    scale_colour_manual(values = c("Near block join" = "#c0392b",
      "Within block" = "#1b1b1b")) +
    labs(x = "Sample-to-sample difference (pA)", y = "Density") +
    theme_channel()
  print(p_seam)
  save_fig(p_seam, "fig5B_seam_test")
} else {
  message("Generated record spans a single block; no joins to test.")
  record_value("ks_seam", "N/A", "single block, no joins")
}

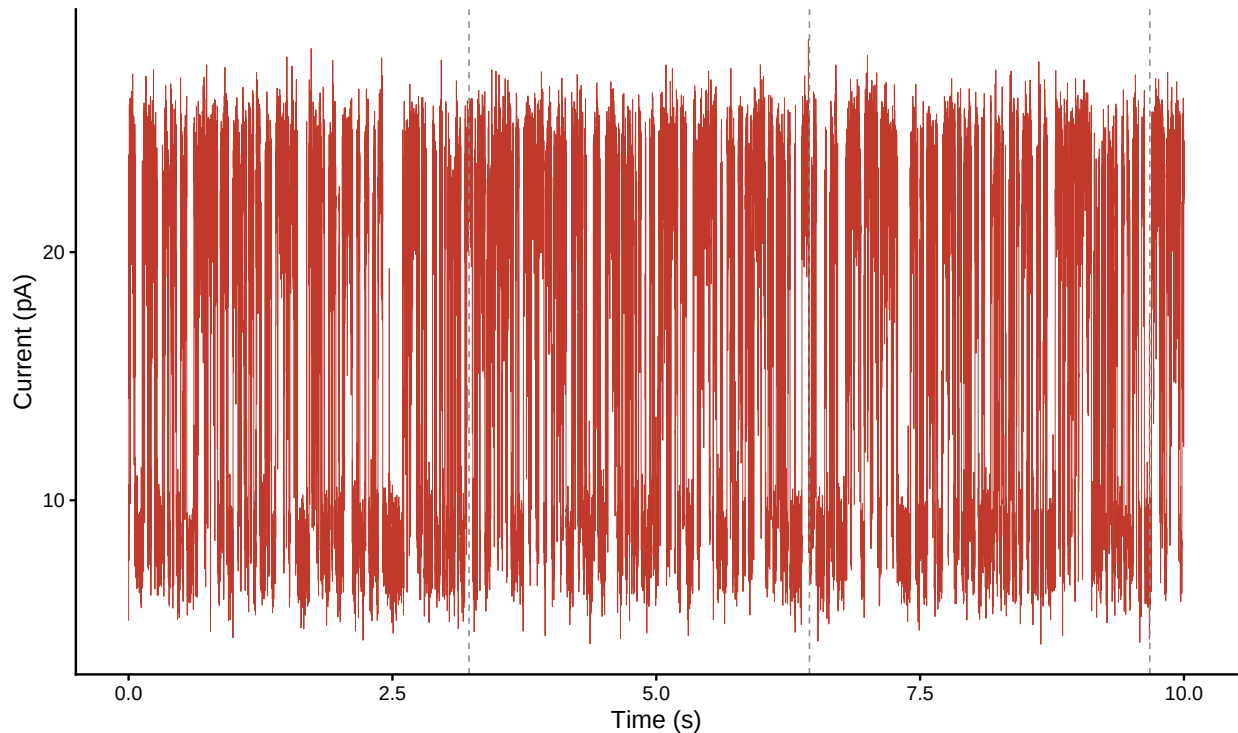
##
## Asymptotic two-sample Kolmogorov-Smirnov test
##
## data: a and b
## D = 0.02038, p-value = 0.555
## alternative hypothesis: two-sided
```



```
# The long trace itself, with boundaries marked.
decim <- max(1L, nrow(gen) %/% 20000L)
long_df <- gen[seq(1, nrow(gen), by = decim), ]

p_long <- ggplot(long_df, aes(time, current)) +
  geom_line(linewidth = 0.15, colour = pal_source[["Generated"]]) +
  { if (length(boundaries))
    geom_vline(xintercept = gen$time[boundaries], linetype = "dashed",
              colour = "grey55", linewidth = 0.3) } +
  labs(x = "Time (s)", y = "Current (pA)") +
  theme_channel()

p_long
```



```
save_fig(p_long, "fig5A_long_record", height = 3)
```

7. Figure 6: dwell times and the ground-truth check

Two comparisons. First, dwell times from the *simulated conditioning idealisation* – this is the exact ground truth. Second, dwell times recovered by idealising the *generated current trace* with a conventional half-amplitude criterion, as though it were an experimental record. The gap between them is the missed-event loss, and being able to measure it at all is the point of the whole conditional design.

```
dw_truth <- dwell_times(gen$channels, FS_GEN) |>
  dplyr::mutate(origin = "Ground truth (simulated)")

n_lev <- max(gen$channels, na.rm = TRUE)
ideal_recovered <- idealise_half_amplitude(gen$current, n_levels = max(1, n_lev))

dw_recov <- dwell_times(ideal_recovered, FS_GEN) |>
  dplyr::mutate(origin = "Recovered from generated current")

dw_real <- dwell_times(real$channels, FS_REAL) |>
  dplyr::mutate(origin = "Real (expert idealised)")

dw_all <- dplyr::bind_rows(dw_truth, dw_recov, dw_real) |>
  dplyr::filter(dwell_ms > 0)

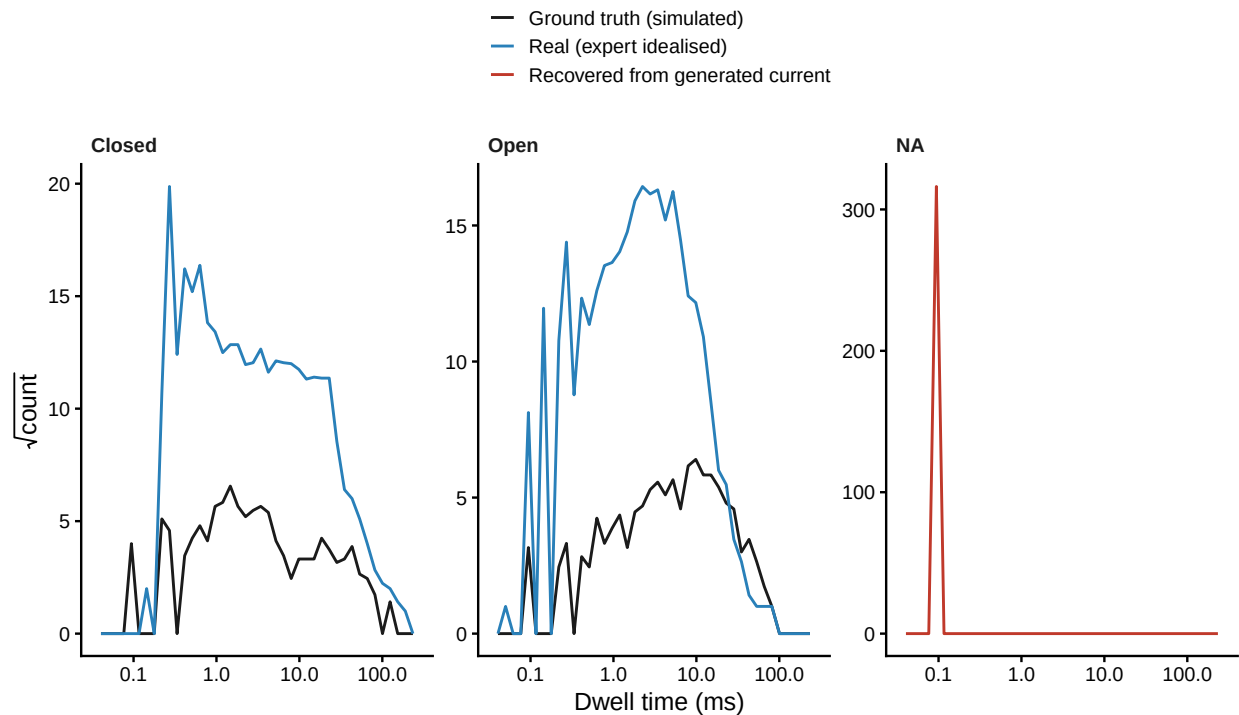
# Sigworth-Sine display: log abscissa, square-root ordinate.
p_dwll <- ggplot(dw_all, aes(dwell_ms, colour = origin)) +
  geom_freqpoly(aes(y = after_stat(sqrt(count))), bins = 40, linewidth = 0.55) +
```

```

scale_x_log10(labels = scales::label_number()) +
facet_wrap(~ state, scales = "free_y") +
scale_colour_manual(values = c(
  "Ground truth (simulated)" = "#1b1b1b",
  "Recovered from generated current" = "#c0392b",
  "Real (expert idealised)" = "#2980b9")) +
labs(x = "Dwell time (ms)", y = expression(sqrt(count))) +
theme_channel() +
theme(legend.position = "top", legend.direction = "vertical")

```

p_dwell



```

save_fig(p_dwell, "fig6_dwell_times", width = 8, height = 4.6)

dwell_summary <- dw_all |>
  dplyr::group_by(origin, state) |>
  dplyr::summarise(
    n = dplyr::n(),
    mean_ms = mean(dwell_ms),
    median_ms = median(dwell_ms),
    .groups = "drop"
  )

knitr::kable(dwell_summary, digits = 3,
  caption = "Dwell-time summary by origin and state.")

```

Table 3: Dwell-time summary by origin and state.

origin	state	n	mean_ms	median_ms
Ground truth (simulated)	Closed	546	8.176	2.00
Ground truth (simulated)	Open	546	10.139	5.85
Real (expert idealised)	Closed	4171	6.268	1.55
Real (expert idealised)	Open	4171	3.610	2.00
Recovered from generated current	NA	100000	0.100	0.10

```
readr::write_csv(dwell_summary, file.path(CFG$out_dir, "dwell_summary.csv"))

get_mean <- function(df, o, s) {
  v <- df |> dplyr::filter(origin == o, state == s) |> dplyr::pull(mean_ms)
  if (length(v)) round(v[1], 3) else NA_real_
}

record_value("tau_open_true",      get_mean(dwell_summary, "Ground truth (simulated)", "Open"))
record_value("tau_closed_true",    get_mean(dwell_summary, "Ground truth (simulated)", "Closed"))
record_value("tau_open_recovered", get_mean(dwell_summary, "Recovered from generated current", "Open"))
record_value("tau_closed_recovered", get_mean(dwell_summary, "Recovered from generated current", "Closed"))
```

Note that `mean_ms` is a mean dwell time, not a fitted exponential time constant. If you want the time constants quoted in the manuscript to be fitted rather than empirical, fit a mixture of exponentials to `dw_all` and substitute those. The empirical mean is a defensible summary and is what the placeholders currently carry.

8. Figure 7: manifold projections

```
Wp_real <- make_windows(real$current, CFG$proj_win, CFG$n_windows_proj)
Wp_gen  <- make_windows(gen$current,  CFG$proj_win, CFG$n_windows_proj)

X <- rbind(Wp_real, Wp_gen)
lab <- rep(c("Real", "Generated"), c(nrow(Wp_real), nrow(Wp_gen)))

# Robust scaling before projection, as in the DeepGANnel pipeline.
X <- scale(X, center = TRUE, scale = TRUE)
X[!is.finite(X)] <- 0

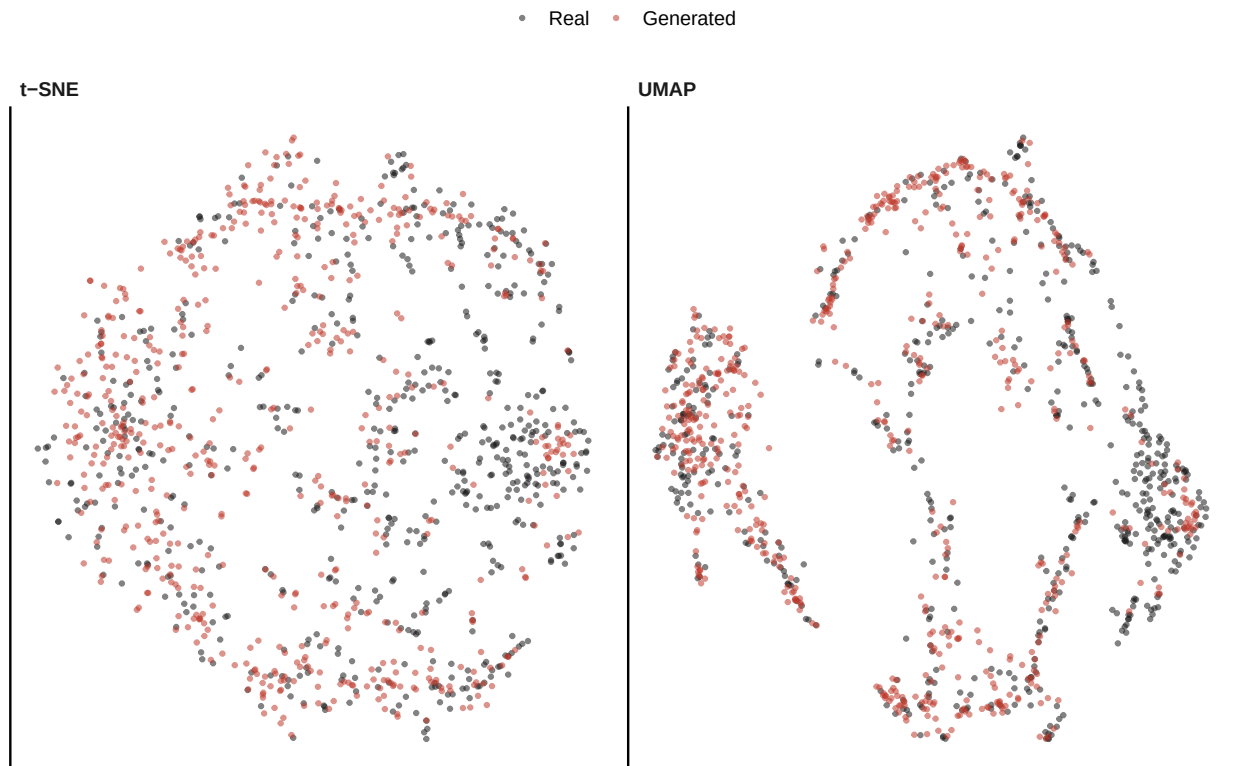
um <- uwot::umap(X, n_neighbors = 15, min_dist = 0.1, n_components = 2)
ts <- Rtsne::Rtsne(X, dims = 2, perplexity = 30, check_duplicates = FALSE,
  pca = TRUE, verbose = FALSE)$Y

proj_df <- dplyr::bind_rows(
  tibble::tibble(x = um[, 1], y = um[, 2], source = lab, method = "UMAP"),
  tibble::tibble(x = ts[, 1], y = ts[, 2], source = lab, method = "t-SNE")
) |>
  dplyr::mutate(source = factor(source, levels = c("Real", "Generated")))

p_proj <- ggplot(proj_df, aes(x, y, colour = source)) +
  geom_point(size = 0.6, alpha = 0.55) +
```

```
facet_wrap(~ method, scales = "free") +
scale_colour_manual(values = pal_source) +
labs(x = NULL, y = NULL) +
theme_channel() +
theme(axis.text = element_blank(), axis.ticks = element_blank())
```

p_proj



```
save_fig(p_proj, "fig7_manifold", width = 8, height = 4.2)

readr::write_csv(proj_df, file.path(CFG$out_dir, "manifold_coordinates.csv"))
```

Cluster separation testing with `ClusterSignificance` (as in `DeepGANnel`) is not run here because the package is a Bioconductor dependency that many installations lack. If you have it, the projection coordinates have been written to `manifold_coordinates.csv` and can be fed to `ClusterSignificance::Pcp()` directly. Record the resulting p value against the `cluster_p` placeholder.

```
record_value("cluster_p", "TO BE FILLED",
             "run ClusterSignificance on manifold_coordinates.csv")
record_value("manifold_description", "REPLACE with reading of Figure 7",
             "describe the degree of overlap you actually see")
```


9. Table 3: collected similarity metrics

```
table3 <- tibble::tibble(
  Metric = c(
    "MMD, real vs generated",
    "MMD, real vs real (split half)",
    "MMD, real vs white noise",
    "KS statistic, amplitude distribution",
    "Closed-state SD, real (pA)",
    "Closed-state SD, generated (pA)",
    "Log10 PSD RMS difference",
    "ACF RMS difference",
    "Envelope SD, real (pA)",
    "Envelope SD, generated (pA)"
  ),
  Value = c(
    signif(mmd_main, 4), signif(mmd_within_real, 4), signif(mmd_white, 4),
    round(unname(ks_amp$statistic), 4),
    round(sd(closed_real), 4), round(sd(closed_gen), 4),
    round(psd_rmse, 3), round(acf_rmse, 4),
    round(sd(env_real), 4), round(sd(env_gen), 4)
  )
)

knitr::kable(table3, caption = "Table 3. Similarity metrics.")
```

Table 4: Table 3. Similarity metrics.

Metric	Value
MMD, real vs generated	0.006371
MMD, real vs real (split half)	0.007015
MMD, real vs white noise	0.013170
KS statistic, amplitude distribution	0.185700
Closed-state SD, real (pA)	1.579000
Closed-state SD, generated (pA)	1.675500
Log10 PSD RMS difference	0.397000
ACF RMS difference	0.132500
Envelope SD, real (pA)	6.166400
Envelope SD, generated (pA)	6.502000

```
readr::write_csv(table3, file.path(CFG$out_dir, "table3_metrics.csv"))
```

10. Write the placeholder values

```
results_tbl <- dplyr::bind_rows(RESULTS$rows) |> dplyr::arrange(placeholder)

readr::write_csv(results_tbl, file.path(CFG$out_dir, "results_values.csv"))
```

```
knitr::kable(results_tbl,
  caption = "Values keyed to the [[placeholders]] in the manuscript draft.")
```

Table 5: Values keyed to the [[placeholders]] in the manuscript draft.

placeholder	value	note
acf_description	ACF curves differ by an RMS of 0.132 over the first 10.0 ms of lag.	REPLACE with real reading of Figure 3B
acf_rmse	0.1325	RMS difference in ACF
block_len	32768	
burst_tc	1000	flicker threshold for the burst clock
cluster_p	TO BE FILLED	run ClusterSignificance on manifold_coordinates.csv
context_len	512	
emb_dim	4	
emb_drop	0.1	
env_sd_cascade	6.502	
env_sd_real	6.166	
env_sd_single	NOT RUN	no single-stage ablation available
filter_khz	1	
ks_D	0.1857	KS statistic, all-points amplitude, real vs generated
ks_seam	0.0204	KS statistic, sample differences near vs away from block joins
long_len_s	10	
manifold_description	REPLACE with reading of Figure 7	describe the degree of overlap you actually see
mmd_env_cascade	0.005	
mmd_env_single	NOT RUN	no single-stage ablation available
mmd_main	0.006371	
mmd_white	0.01317	
mmd_within	0.007015	
modes_gen	8.11	modal current, generated, pA
modes_real	8.23	modal current, real, pA
n_blocks	4	
n_records	1	seed records
n_samples	824,001	total seed samples
n_seconds	41.2	seconds of seed data
psd_description	Real and generated spectra agree to an RMS difference of 0.40 log10 units below the 1000 Hz filter corner.	REPLACE with real reading of Figure 3A
psd_log_rmse	0.397	RMS difference in log10 PSD below filter corner
sample_khz	20	
sd_gen	1.675	closed-state SD, generated, pA
sd_real	1.579	closed-state SD, real, pA
seq_len	128	
seq_len_ms	6.4	training window duration
slow_factor	64	
tau_closed_recovered	8.176	
tau_closed_train	8.176	

placeholder	value	note
tau_open_recovered	NA	
tau_open_trunk	10.14	

```
cat("\nWritten to", file.path(CFG$out_dir, "results_values.csv"), "\n")
```

```
##
## Written to ./figures/results_values.csv
```

```
cat("Any row reading TO BE FILLED, NOT RUN or REPLACE needs attention",
    "before the manuscript goes anywhere.\n")
```

```
## Any row reading TO BE FILLED, NOT RUN or REPLACE needs attention before the manuscript goes anywhere
```

Session information

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_United Kingdom.utf8
## [2] LC_CTYPE=English_United Kingdom.utf8
## [3] LC_MONETARY=English_United Kingdom.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United Kingdom.utf8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods    base
##
## other attached packages:
## [1] patchwork_1.3.2 arrow_25.0.0   purrr_1.2.2   readr_2.2.0
## [5] ggplot2_4.0.3  tidyr_1.3.2    dplyr_1.2.1
##
## loaded via a namespace (and not attached):
## [1] Matrix_1.7-4      bit_4.6.0      gtable_0.3.6    crayon_1.5.3
## [5] compiler_4.5.2    Rcpp_1.1.2      tidyselect_1.2.1 FNN_1.1.4.1
## [9] parallel_4.5.2    assertthat_0.2.1 scales_1.4.0     yaml_2.3.12
## [13] fastmap_1.2.0     uwot_0.2.4      lattice_0.22-7  R6_2.6.1
## [17] labeling_0.4.3     generics_0.1.4  knitr_1.51      Rtsne_0.17
```

## [21] tibble_3.3.1	pillar_1.11.1	RColorBrewer_1.1-3	tzdb_0.5.0
## [25] rlang_1.3.0	xfun_0.60	S7_0.2.2	bit64_4.8.2
## [29] cli_3.6.6	withr_3.0.3	magrittr_2.0.5	digest_0.6.39
## [33] grid_4.5.2	vroom_1.7.1	hms_1.1.4	lifecycle_1.0.5
## [37] vctrs_0.7.3	RSpectra_0.16-2	evaluate_1.0.5	glue_1.8.1
## [41] farver_2.1.2	rmarkdown_2.31	tools_4.5.2	pkgconfig_2.0.3
## [45] htmltools_0.5.9			